COMP 1111 2. Midterm

Olcay Taner YILDIZ

I. Question (20 points)

Trace the execution of the call mystery(4) for the following recursive method by drawing method calls and return values.

```
public static int mystery(int n) {
   if (n <= 0)
      return 0;
   else
      return n + mystery(n - 1) +
            mystery(n - 3);
}
```

II. Question (20 points)

Write a program which takes two integers a and b from the user. Then the program will randomly generate b - a + 1 numbers between [a, b] and print the number of integers

- which are divisible by 2 but not by 3 and 5.
- which are divisible by 2 and 3 but not by 5.
- which are divisible by all three numbers
- which are not divisibly by any three.

```
a: 19
b: 31
Divisible by 2 but not by 3 and 5: 3
Divisible by 2 and 3 but not by 5: 1
Divisible by all: 1
Divisible by none: 4
```

III. Question (20 points)

Write a program which will randomly generates 10.000 four digit valid passwords and prints the largest one. The rules for a valid password are:

- Any three digits can not be same.
- It can not be your birth year (Use your birth year in this question).
- The digits can not be sequential such as 1234, 2345, 6543 etc.

```
Largest random password: 9981
```

IV. Question (20 points)

- Write the function **dice**

  `dice()`

  which randomly throws a dice and returns the result.
- Write the function **pair**

  `pair(int dice1, int dice2, int dice3)`

  which returns true if there are at least two dices whose values are equal, false otherwise.
- Write the function **triple**

  `triple(int dice1, int dice2, int dice3)`

  which returns true if all three dices are equal, false otherwise.
- Write the function **diceGame**

  `diceGame()`

  which plays the following dice game using the above functions. There are two players. The first player throws three dices. The second player throws three dices. Then the dices of players are compared according to this rule: A triple in three dices is better than a pair in three dices which is better than nothing.

```
First player: 1 2 1
Second player: 5 5 5
Second player wins

First player: 1 5 6
Second player: 4 1 2
No one wins
```

V. Question (20 points)

Let say you have the function **isPrime**

```
public static bool isPrime(int N)
```

which returns true is N is prime, false otherwise. DO NOT WRITE THIS FUNCTION.

Write the first function **largestPower**

```
public static int largestPower(int N, int x)
```

which returns the largest power of x, which divides N. For example, degree(5040, 2) returns 4, because 2^4 is the largest power of 2 which divides 5040. Another example, degree(120, 3) returns 1, because 3^1 is the largest power of 3 which divides 120.

Write the second function **primeDivisors**

```
public static void primeDivisors(int N)
```

which takes input an integer N and prints the prime divisors of N and their largest powers which divide N. You must use the isPrime and largestPower functions given above in **primeDivisors**.

```
N = 120
2 3
3 1
5 1

N = 5040
2 4
3 2
5 1
7 1
```